NykaaSight Hackathon Pitch Deck (Text Slides)
Prepared on March 11, 2026

==
SLIDE 1 - Title
==
NykaaSight
Conversational AI for Instant Business Intelligence Dashboards

Tagline:
Ask a business question in plain English.
Get an interactive dashboard in seconds.

==
SLIDE 2 - The Real Problem
==
Business teams have data, but insight access is slow.

Current pain:
1. Non-technical users depend on analysts for basic reports.
2. BI tools are powerful but require setup and technical fluency.
3. Quick executive questions take days instead of minutes.

Result:
Decision-making slows down because insight delivery is bottlenecked.

==
SLIDE 3 - Our Solution
==
NykaaSight is an AI-powered BI copilot for non-technical users.

User flow:
1. Upload CSV data (or use demo dataset).
2. Ask business questions in natural language.
3. Receive a dashboard with multiple interactive charts and insights.
4. Continue with follow-up prompts to refine charts.

==
SLIDE 4 - Who It Is For
==
Primary persona: Non-Technical Executive (CXO)

What they want:
1. Fast and trustworthy metrics.
2. No SQL.
3. No manual chart configuration.

NykaaSight gives:
1. Conversational interface.
2. Auto chart selection.
3. Data-backed insights with visual context.

==
SLIDE 5 - System Architecture
==
Frontend:
- React + Vite
- Recharts visualization engine

Backend:
- Node.js + Express REST APIs
- Data processor and LLM orchestration

Intelligence:
- NVIDIA NIM LLM + embeddings
- Vector retrieval for contextual memory

Storage:
- Supabase PostgreSQL
- Tables for datasets, sessions, messages, embeddings, snapshots

==
SLIDE 6 - Upload Pipeline (What Happens Internally)
==
When user uploads CSV:
1. Backend receives file via multer memory upload.
2. CSV is parsed into rows.
3. Column types are inferred:
   - number, date, categorical, text
4. Dataset metadata is saved in Supabase.
5. Full rows are cached in-memory for fast chart processing.
6. Background vector indexing creates semantic chunks for retrieval.

Why this matters:
Chat responses become faster and more context-aware.

==
SLIDE 7 - Chat-to-Dashboard Pipeline
==
For every chat prompt:
1. Backend gets dataset rows from cache.
2. Vector service retrieves relevant memory context from embeddings.
3. LLM receives:
   - schema
   - sample rows
   - prior context
   - recent chat history
   - current dashboard state
4. LLM returns strict JSON:
   - reply text
   - chart configs
   - follow-up suggestions
5. Backend processes chart configs into chart-ready datasets.
6. Frontend renders charts in Recharts.
7. Messages and chart memory are persisted.

==
SLIDE 8 - Chart Intelligence
==
Chart processor supports:
1. Filters:
   - equality
   - range (gte/lte)
   - list (in)
2. Grouping by category.
3. Aggregations:
   - sum, avg, count, max, min

4. Sort and limit.
5. Formatting for each chart type:
   - bar, line, area, pie, scatter, composed

Reliability hardening:
- Chart type normalization handles model variants (pie, pie chart, donut).
- Pie fallback logic ensures slices render even with imperfect configs.

==
SLIDE 9 - UX and Product Features
==
Key UX capabilities:
1. Upload states: idle -> uploading -> indexing -> done.
2. Suggested prompt chips for first-time users.
3. Loading states and error toasts.
4. KPI row with animated counters.
5. Chart controls: switch type, fullscreen, delete.
6. Save and restore dashboard snapshots.
7. Print/export dashboard view.
8. History page for datasets and sessions.

==
SLIDE 10 - Accuracy Strategy
==
How we improve correctness:
1. Structured system prompt enforces strict JSON output.
2. Prompt includes actual schema and sample rows.
3. Vector retrieval injects relevant prior context.
4. Backend computes metrics directly from dataset rows.
5. If parsing fails, user gets safe and clear retry guidance.

Net effect:
Lower hallucination risk and stronger data grounding.

==
SLIDE 11 - Evaluation Rubric Mapping
==
Accuracy (40):
- Data retrieval and aggregation run on real dataset rows.
- Chart type rules plus normalization improve chart relevance.
- Clear fallback handling for parsing and dataset edge cases.

Aesthetics and UX (30):
- Professional dashboard interface.
- Interactive charts with tooltips and legends.
- Smooth flow with loading states, toasts, and snapshots.

Approach and Innovation (30):
- Text -> LLM -> data processor -> chart renderer pipeline.
- Retrieval-augmented context via embeddings.
- Stateful conversational updates to existing dashboards.

==
SLIDE 12 - Bonus Criteria Coverage
==
Follow-up Questions:
- Supported.
- Users can refine existing charts (filter, modify, remove) conversationally.

Data Format Agnostic Upload:
- Supported for CSV.
- Users can upload their own data and start prompting immediately.

==
SLIDE 13 - Demo Script (10 Minutes)
==
Minute 1:
- Open app and load demo dataset.

Minutes 2-4 (Prompt 1, basic):
"Show monthly revenue trend and total conversions by month."

Minutes 4-6 (Prompt 2, intermediate):
"Compare ROI by Channel_Used and show top campaign types by conversions."

Minutes 6-8 (Prompt 3, advanced follow-up):
"Now keep top 5 channels, remove weak performers, and show segment-wise revenue share."

Minutes 8-10:
- Save snapshot.
- Open history and restore previous session.
- Export dashboard view.

==
SLIDE 14 - What Is Production-Ready vs Prototype
==
Already implemented:
1. End-to-end conversational dashboard generation.
2. Multi-page polished frontend.
3. Data persistence and vector memory.
4. Error handling and fallback logic.

Next production upgrades:
1. Auth and user-level dataset isolation.
2. Access control and enterprise governance.
3. Cloud deployment with monitoring and autoscaling.
4. Expanded connectors beyond CSV.

==
SLIDE 15 - Final Pitch Close
==
NykaaSight transforms BI from tool-driven to conversation-driven.

Business value:
1. Faster executive decisions.
2. Lower analyst dependency for recurring reporting.
3. Better data democratization across teams.

One-line close:
"If you can ask it, you can dashboard it."